

CryptoBroker: отчет и аналитика решений

Система защищенного обмена сообщениями с криптоконтейнерами

Сводка в цифрах

Показатель	Значение
Файлов frontend	31
Файлов backend .mjs	2
Файлов LiteBroker	3
Кодовых файлов всего	36
Строк кода примерно	4797
Строк в App.tsx	360
Строк в server/index.mjs	594
Строк в server/crypto.mjs	47

1. Что представляет собой проект

CryptoBroker - это локальная система защищенного обмена сообщениями. Проект состоит из основной веб-версии CryptoBrokerWeb и облегченной версии LiteBroker. Основная версия показывает пользовательский сценарий мессенджера: регистрация, вход, поиск пользователей по username, создание личного чата, отправка сообщений и просмотр криптоконтейнера. LiteBroker оставлен как легкая демонстрация идеи защищенной передачи сообщений без тяжелого интерфейса.

Проект специально не использует Telegram API, реальные аккаунты, номера телефонов и внешние сервисы авторизации. Это важно для безопасности: приложение не имитирует чужой сервис и не собирает реальные пользовательские данные.

2. Почему выбран именно такой формат

Тема проекта связана с обменом сообщениями с криптографической защитой информации. Поэтому главным защищаемым объектом выбрано сообщение, а не абстрактное значение датчика. На защите проще показать цепочку: пользователь пишет текст, backend создает криптоконтейнер, база хранит защищенное содержимое, интерфейс показывает технические поля контейнера.

Кафка и IoT-подход полезны для объяснения брокерной архитектуры, но для темы мессенджера они являются вспомогательными. Поэтому основная версия сделана как мессенджер, а облегченная версия показывает идею защищенной передачи событий отдельно.

3. Архитектура

Архитектура построена с разделением ответственности. React отвечает только за интерфейс. Node.js API принимает запросы, проверяет пользователя, управляет чатами и сообщениями. SQLite хранит локальные данные. Отдельный crypto layer создает и расшифровывает AES-256-GCM контейнеры. Prometheus и Grafana используются для наблюдаемости системы.

Такое разделение уменьшает поверхность атаки: frontend не имеет прямого доступа к базе данных, ключам и криптографическим операциям. Все операции проходят через backend и контролируемые маршруты API.

4. Что обновлено в последней версии

Криптографическая логика вынесена из server/index.mjs в отдельный модуль server/crypto.mjs. Это делает архитектуру понятнее: основной backend управляет маршрутами и БД, а crypto-модуль отвечает за создание и проверку криптоконтейнеров.

В интерфейсе разделены режимы поиска: Чаты и Пользователи. Теперь поиск по username не смешивается со списком чатов, а пользователь видит понятный сценарий: найти username и начать чат.

Добавлены frontend-хуки useSettings и useKeyboardShortcuts. Это разгружает App.tsx и показывает движение к более чистой компонентной архитектуре.

Добавлен файл docs/design-system.md с описанием токенов, компонентов, экранов для Figma и выбранного грубого secure-стиля.

5. Криптографическая защита

Для сообщений используется AES-256-GCM. Этот режим выбран потому, что он одновременно обеспечивает конфиденциальность и контроль целостности. Контейнер содержит version, algorithm, iv, ciphertext, authTag и metadata.

Новые сообщения не должны храниться в базе как открытый текст. В базе сохраняется cryptoContainer, а расшифрованный текст используется только для отображения в интерфейсе. Панель Крипто в правой части мессенджера позволяет показать контейнер на защите.

Пароли пользователей не хранятся открыто. Для них используется PBKDF2-хеширование с солью. Это базовая, но понятная мера защиты для учебного проекта.

6. Почему нужны Grafana и Prometheus

Grafana и Prometheus не нужны обычному пользователю для переписки. Они нужны администратору и для защиты проекта: показывают, что система наблюдаемая. Prometheus собирает метрики backend, а Grafana отображает их на дашбордах.

Это связано с security-by-design: защищенная система должна быть не только шифрующей, но и наблюдаемой. Метрики помогают видеть доступность API, ошибки и подозрительную активность.

7. UML-диаграммы

UML и Mermaid-схемы лежат в CryptoBrokerWeb/docs/architecture.md. Там описаны компонентная схема, sequence diagram, class diagram и связи между frontend, backend, БД, crypto layer, Prometheus и Grafana.

Диаграммы нужны не как украшение, а как доказательство архитектурного подхода: видно, какие компоненты изолированы, где проходит политика доступа и где создается криптоконтейнер.

8. Паттерны проектирования

Facade: frontend работает с api.ts как с простым фасадом и не знает деталей fetch, маршрутов и ошибок backend.

Repository: работа с SQLite отделена от пользовательского интерфейса и сосредоточена на серверной стороне.

DTO: backend отдает наружу только безопасные представления объектов. Например, парольные хеши не уходят во frontend.

Strategy: криптографический слой можно заменить или расширить другим алгоритмом без переписывания интерфейса.

Observer: polling и метрики отражают наблюдение за состоянием системы. Следующий шаг - WebSocket или Server-Sent Events.

Mediator: API выступает посредником между UI, базой, криптографией и мониторингом.

9. Шаблоны защиты

Минимизация поверхности атаки: frontend не работает напрямую с БД и ключами.

Изоляция компонентов: Docker и отдельные сервисы позволяют запускать приложение, мониторинг и легкую версию раздельно.

Безопасные значения по умолчанию: нет реальных внешних аккаунтов, нет Telegram Login, нет сбора номеров телефонов.

Контроль целостности: AES-GCM authTag позволяет обнаружить изменение контейнера.

Наблюдаемость: Prometheus и Grafana позволяют контролировать состояние системы.

10. Ограничения и развитие

Учебная версия не является промышленным аналогом Signal. Следующие улучшения: WebSocket вместо polling, httpOnly cookie вместо localStorage, audit_log для событий безопасности, rate limit на login/register, клиентское E2E-шифрование и разнос backend по routes/db/auth/metrics модулям.

Текущая версия уже показывает базовую цепочку защищенного обмена: пользователь - UI - API - crypto layer - SQLite - monitoring.

Сравнение с аналогами

Аналог	Сильная сторона	Почему не выбран как основа
Telegram Web	Удобный UX, привычный формат чатов	Нельзя использовать API/бренд/логин; закрытая инфраструктура не показывает нашу БД и криптослой
Signal	Сильная E2E-криптография	Сложнее для учебной демонстрации и модификации
Matrix/Element	Открытый протокол и федерация	Слишком тяжелая инфраструктура для учебной защиты
Kafka-only	Хорош для брокерных событий	Нет полноценного пользовательского UX мессенджера
MQTT/IoT	Простая телеметрия датчиков	Хуже соответствует теме защищенной переписки
CryptoBroker	Контроль UI, backend, БД, криптослоя и мониторинга	Учебная версия; требует дальнейшего усиления до промышленного уровня

Структура проекта

Путь	Назначение
CryptoBrokerWeb/src	React + TypeScript интерфейс мессенджера
CryptoBrokerWeb/src/hooks	Вынесенные frontend-хуки: настройки и горячие клавиши
CryptoBrokerWeb/src/utlis/api.ts	Единый API-фасад, ошибки и запросы к backend
CryptoBrokerWeb/server/index.mjs	Node.js API, маршруты, SQLite, авторизация
CryptoBrokerWeb/server/crypto.mjs	Создание, расшифровка и проверка AES-256-GCM криптоконтейнеров
CryptoBrokerWeb/server/data/cryptobroker.sqlite	Локальная база данных пользователей, чатов, сообщений и сессий
CryptoBrokerWeb/docs/architecture.md	UML/Mermaid-диаграммы архитектуры
CryptoBrokerWeb/docs/design-system.md	Токены, компоненты, Figma-экраны и UI-логика
LiteBroker	Легкая отдельная версия с отдельной БД
artifacts	Отчет, презентация, PDF и дополнительные материалы

Итоговое обоснование

Выбранные решения дают баланс между понятной демонстрацией и реальной инженерной логикой. React + TypeScript показывает рабочий интерфейс, Node.js API изолирует бизнес-логику, SQLite упрощает локальный запуск, AES-256-GCM показывает криптографическую защиту сообщений, а Prometheus/Grafana добавляют наблюдаемость.